QUIC Setup: Frontend Server and Game Server

Oct 18, 2026 · @Prakhar

This guide builds two Rust servers that talk over QUIC: a frontend server that meets players, and a game server that runs the games. Browsers connect with WebTransport (QUIC for the web), with WebSocket as a backup. I have not compiled the code below, so treat it as a starting sketch and check each crate's documentation.

1. The two servers

	Frontend server	Game server
Job	Meets players, hosts the web page, forwards requests to the game server, sends replies back	Runs the games: checks moves, runs clocks, saves moves
Parts from the architecture document	Gateway	Game Service and Persistence Writer, with Redis, PostgreSQL and NATS next to it
Who connects to it	Players' browsers (public)	Only the frontend server (private)
Ports	UDP 443 for WebTransport, TCP 443 for the web page and WebSocket	UDP 4434, open only to the frontend server
How it grows	Add more frontend servers	Add shards, then more game servers

Why two servers? The frontend server holds the many player connections and does the encryption work. The game server keeps the games in memory. A crash or a traffic spike on one does not have to take down the other, and the game server is never exposed to the internet.

2. The big picture

1. The browser opens a WebTransport connection to the frontend server.

2. The frontend server keeps one QUIC connection to the game server. On it, it keeps one long-lived stream for each shard.
3. When a player sends a move, the frontend server wraps it in a `Request` and writes it to the stream of shard `session_id % N`.
4. The game server checks the move and writes a `Reply` back on the same stream.
5. The frontend server finds the player by `conn_id` and sends the reply to that browser.
6. News for the other player and for spectators comes back the same way, as `Update` messages.

The next sections build each piece in order.

3. Before you start

WebTransport is available in every major browser. It became Baseline "newly available" on 2026-03-24, when Safari added it, according to the web-features data.

Browser	First version with WebTransport
Chrome and Edge	97
Firefox	114
Safari on macOS and iOS	26.4 (released 2026-03-24)

A third-party compatibility page says the operating system must also be 26.4 or newer for Safari, and that support inside in-app web views is unconfirmed. Test on real devices.

You need these tools:

Tool	What it is used for
Rust and tokio	The language and the async runtime
wtransport (version 0.6.1 when I checked)	A WebTransport server and client in pure Rust, built on the QUIC library quinn
A web server library or nginx	Serves the web page over HTTPS and handles the WebSocket backup
`serde` with a compact format such as `bincode` or `postcard`	Turns messages into bytes and back
A domain name and a certificate	Browsers must trust the frontend server

4. Step 1: certificates

QUIC always uses TLS, so every server needs a certificate. You need three kinds:

- **Public certificate for the frontend server.** Get a free one from Let's Encrypt for your domain name. Browsers refuse to connect when they do not trust the certificate.
- **Test certificate on your computer.** Make a self-signed certificate and give its hash to the browser with `serverCertificateHashes`. The key must be ECDSA (the specification rejects RSA for this), and the certificate must be short-lived. The specification allows at most 14 days. For quick tests you can also start Chrome with `--webtransport-developer-mode`, as the compatibility page notes.
- **Private certificate for the game server.** The frontend server trusts only this one, so nobody else can pretend to be the game server.

Create a test certificate and print its hash:

```
openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:prime256v1 \
  -days 13 -nodes -keyout key.pem -out cert.pem \
  -subj "/CN=localhost" -addext "subjectAltName=DNS:localhost"

openssl x509 -in cert.pem -outform der | openssl dgst -sha256 -binary |
base64
```

5. Step 2: build the frontend server

The frontend server accepts WebTransport sessions from browsers, reads each message from the player's stream, and forwards it to the game server.

```
use wtransport::{Endpoint, Identity, ServerConfig};

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let identity = Identity::load_pemfiles("cert.pem", "key.pem").await?;
    let config = ServerConfig::builder()
        .with_bind_default(443)          // UDP port for browsers
        .with_identity(identity)
        .build();
    let server = Endpoint::server(config)?;

    loop {
        let incoming = server.accept().await;
        tokio::spawn(async move {
            let request = incoming.await?;           // the browser's connect
request
```

```
            let connection = request.accept().await?;
            let conn_id = next_conn_id();            // your own counter

            // one stream per player for moves
            let (send, mut recv) = connection.accept_bi().await?;
            register_player(conn_id, send);          // so replies can find
this player

            loop {
                let frame = read_frame(&mut recv).await?;
                let msg = decode_from_player(&frame)?;
                // build a Request with conn_id, then send it to the game
server
                forward_to_game_server(conn_id, msg).await?;
            }
            #[allow(unreachable_code)]
            Ok::<_, anyhow::Error>(())
        });
    }
}
```

Also run a normal HTTPS server on TCP 443. It serves the web page and the WebSocket backup, and it uses the same `forward_to_game_server` function, so both kinds of players take the same path.

6. Step 3: build the game server

The game server listens on a private UDP port. It accepts one connection from the frontend server, checks the first message, and then reads the shard streams.

```
use wtransport::{Endpoint, Identity, ServerConfig};

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let identity = Identity::load_pemfiles("game-cert.pem", "game-
key.pem").await?;
    let config = ServerConfig::builder()
        .with_bind_default(4434)         // private UDP port
        .with_identity(identity)
        .keep_alive_interval(Some(std::time::Duration::from_secs(3)))
        .build();
    let server = Endpoint::server(config)?;

    loop {
        let incoming = server.accept().await;
```

```
        tokio::spawn(async move {
            let connection = incoming.await?.accept().await?;

            // the first stream carries Hello
            let (mut send, mut recv) = connection.accept_bi().await?;
            check_hello(&mut recv, &mut send).await?;   // version and secret
token

            // then one stream per shard
            loop {
                let (send, recv) = connection.accept_bi().await?;
                tokio::spawn(serve_shard_stream(send, recv));
            }
            #[allow(unreachable_code)]
            Ok::<_, anyhow::Error>(())
        });
    }
}
```

`serve_shard_stream` reads each frame, decodes a `ToGame` message and puts it in the inbox of its shard. A second task takes replies from the shard and writes them back on the same stream. Both are the shard inboxes from the architecture document.

7. Step 4: how the two servers talk

7.1 Rules

- **One connection.** The frontend server opens one QUIC connection to the game server and keeps it open.
- **One stream per shard.** Stream `k` carries all requests for shard `k`. Requests for one game always use the same stream, so they stay in order.
- **Frames.** A stream is only a flow of bytes, so every message is sent as a frame: 4 bytes for the length, then the message.
- **Hello first.** The frontend server sends `Hello` with a protocol version and a secret token. The game server answers `HelloOk` and tells it how many shards (`N`) there are.
- **Keep-alive.** Both sides send QUIC keep-alive packets, so a dead connection is noticed within seconds.
- **Limits.** Reject any frame bigger than 64 KB, and use bounded queues so a slow game server cannot fill the frontend server's memory.

7.2 Frame helpers

These work with any stream that supports tokio's `AsyncRead` and `AsyncWrite`. Check that your version of wtransport implements them.

```
use tokio::io::{AsyncRead, AsyncReadExt, AsyncWrite, AsyncWriteExt};

pub async fn write_frame<W: AsyncWrite + Unpin>(w: &mut W, bytes: &[u8]) ->
anyhow::Result<()> {
    w.write_all(&(bytes.len() as u32).to_be_bytes()).await?;
    w.write_all(bytes).await?;
    Ok(())
}

pub async fn read_frame<R: AsyncRead + Unpin>(r: &mut R) ->
anyhow::Result<Vec<u8>> {
    let mut len = [0u8; 4];
    r.read_exact(&mut len).await?;
    let len = u32::from_be_bytes(len) as usize;
    anyhow::ensure!(len <= 64 * 1024, "frame too big");
    let mut buf = vec![0u8; len];
    r.read_exact(&mut buf).await?;
    Ok(buf)
}
```

7.3 Messages

```
pub enum ToGame {
    Hello { version: u16, token: String },
    Join { conn_id: u64, session_id: u64, role: u8 }, // a player or
spectator joins a game
    Request(Request),                                  // from the
architecture document
    Leave { conn_id: u64 },
}

pub enum FromGame {
    HelloOk { version: u16, shards: u16 },
    Reply(Reply),                                      // answer to one
Request
    Update { session_id: u64, state: Vec<u8> },        // news for everyone
watching a game
    Error { conn_id: u64, code: u16 },
}
```

Message	Sent by	Used for
`Hello`, `HelloOk`	Frontend, game server	Check the version and the secret, and share the shard count
`Join`, `Leave`	Frontend server	Tell the game server who is watching, so it can address updates
`Request`	Frontend server	A select or a move from a player
`Reply`	Game server	The answer to one request, sent to the player who asked
`Update`	Game server	A move for the other player and the spectators. The frontend server fans it out
`Error`	Game server	A refused request, such as an illegal move

7.4 Stronger security between the servers

The simple setup is a private certificate, a firewall that allows only the frontend server, and the secret token in `Hello`. For more, use mutual TLS: the game server also checks a certificate from the frontend server. wtransport exposes the quinn library underneath, and quinn can do this through its TLS settings. Check the crate documentation for the exact setup.

8. Step 5: connect the web page

```
async function connect() {
  if ("WebTransport" in window) {
    try {
      const wt = new WebTransport("https://chess.example.com/play");
      await wt.ready;
      const stream = await wt.createBidirectionalStream();
      return {
        kind: "webtransport",
        writer: stream.writable.getWriter(),
        reader: stream.readable.getReader(),
      };
    } catch (e) {
      console.warn("WebTransport failed, using WebSocket", e);
    }
  }
  const ws = new WebSocket("wss://chess.example.com/play");
  await new Promise((ok, fail) => { ws.onopen = ok; ws.onerror = fail; });
  return { kind: "websocket", socket: ws };
}
```

- A WebTransport stream is bytes, so use the same frames as in section 7.2. A WebSocket message is already a whole message, so it needs no length.
- For local tests with a self-signed certificate, pass `serverCertificateHashes: [{ algorithm: "sha-256", value: hashBytes }]` as the second argument to `new WebTransport(...)`.
- Browser developer tools did not show WebTransport traffic as of July 2026, so log frames in your code on both sides.

9. Step 6: run and test on your computer

1. Create the test certificate (section 4) and the game server's own certificate.
2. Start the game server: `cargo run --bin game-server`.
3. Start the frontend server: `cargo run --bin frontend-server`. It should connect to the game server and log `HelloOk` with the shard count.
4. Open the web page in Chrome, with the certificate hash set or with `--webtransport-developer-mode`.
5. Send a move. Check that the frontend log shows the `Request`, the game server log shows the `Reply`, and the page shows the answer.

6. Block UDP on the frontend port, reload, and check that the page falls back to WebSocket.
7. Stop the game server and start it again. Check that the frontend server reconnects and the next move works.

10. Step 7: run both servers on the free Oracle VM

The free hosting plan gives 2 Arm CPUs and 12 GB in total, and Oracle allows one Arm VM with 2 CPUs or two Arm VMs with 1 CPU each. You have two choices:

Choice	How	Good for
One VM	Run both servers on it. The game server listens only on `127.0.0.1` or a private port	The simplest start
Two VMs	One VM per server, 1 CPU each. The game server accepts UDP 4434 only from the frontend VM's private address	Keeping the servers apart, with less CPU each

In both cases, open UDP 443 and TCP 443 to the internet for the frontend server, in the network security list and in the VM firewall. Never open the game server's port to the internet.

11. What happens when something breaks

Problem	What happens	What to do
The connection between the servers drops	Requests in flight are lost	The frontend server reconnects with longer and longer waits, sends `Join` again for every active player, and the client retries with the same `seq`, so repeats are ignored
The game server restarts	Games are loaded again from storage when their next request arrives	Same as above
The frontend server restarts	Players' connections drop	The page reconnects and asks for the game state

Problem	What happens	What to do
UDP is blocked on the player's network	WebTransport fails. The compatibility page says Chrome and Firefox have no HTTP/2 fallback	The page falls back to WebSocket
The certificate is wrong or expired	The browser refuses the session	Check the name, the expiry date and the key type
A port is closed	Connections time out	Open UDP in both the security list and the VM firewall

12. Sources

Checked on the date at the top.

- web-features: WebTransport
- WebTransport compatibility notes (third-party)
- wtransport crate documentation
- Oracle Cloud: Always Free Resources
- kmcd.dev: WebTransport notes (third-party)